

Progetto 2: Analisi in tempo reale di dati sui voli con Apache Flink

Sistemi e Architetture per Big Data - A.A. 2025/26

Luca Carloni
0366574

Alessandro Vassallo
0366572

Andrea Galluzzi
0365026

Sommario—Il presente lavoro propone un’architettura di stream processing, incentrata sul framework Apache Flink, per l’analisi in tempo reale di un dataset di circa 2,2 milioni di eventi relativi ai voli aerei statunitensi (gennaio–aprile 2025). Il dataset storico viene trattato come uno stream mediante una fase di replay accelerato: un producer dedicato pubblica gli eventi su Apache Kafka rispettando l’ordine di event time e iniettando in modo controllato eventi fuori ordine, serializzandoli in formato Apache Avro tramite un Confluent Schema Registry. Su tale flusso vengono calcolate tre query su finestre temporali basate sull’event time, con gestione del disordine tramite watermarking; per la stima dei percentili (Q3) si adotta lo sketch approssimato e mergeable DDSketch. Come richiesto per i gruppi da tre persone, si implementa un meccanismo di rilevamento di eventi fuori ordine e si confrontano più configurazioni di watermarking sul piano di completezza e latenza; l’intera soluzione è inoltre reimplementata in Apache Spark Structured Streaming per un confronto tra i due motori. Si realizza infine la tolleranza ai guasti degli operatori con stato tramite il meccanismo di checkpointing di Flink, con verifica sperimentale del recupero dello stato. I risultati sono consegnati in formato CSV e visualizzati in tempo reale su Grafana, con due backend di storage alternativi (InfluxDB e TimescaleDB).

Index Terms—Apache Flink, Apache Kafka, Apache Avro, Confluent Schema Registry, Apache Spark, DDSketch, checkpointing, InfluxDB, TimescaleDB, Grafana

I. INTRODUZIONE

L’obiettivo del progetto è di sviluppare un’architettura di stream processing, con particolare attenzione al framework di data processing **Apache Flink**, per rispondere ad alcune query su dati storici relativi a voli aerei statunitensi, forniti dal Dipartimento dei trasporti degli Stati Uniti d’America. Il dataset contiene circa 2.200.000 eventi, che descrivono i voli effettuati nel periodo dal 1 gennaio 2025 al 30 aprile 2025. La quantità di dati forniti non giustifica appieno l’utilizzo di framework Big Data, ma le scelte progettuali e di sviluppo sono state guidate dalla volontà di simulare un contesto dove è necessario avere questo tipo di approccio. A differenza dell’analisi batch vista nel *Progetto 1: Analisi di dati sui voli con Apache Spark*, dove l’intero dataset è stato processato a posteriori, l’analisi in tempo reale richiede che gli eventi vengano elaborati man mano che vengano prodotti, mantenendo aggregazioni aggiornate su finestre temporali, gestendo possibili ritardi e disordine nell’arrivo degli eventi.

Oltre alle tre query richieste sono state anche sviluppate le estensioni previste dalla specifica: un meccanismo di rilevamento di eventi fuori ordine con il confronto di più configurazioni di watermarking e una reimplementazione dell’intera soluzione in *Apache Spark Structured Streaming* ai fini di confronto tra i due motori di elaborazione. Inoltre, sono stati realizzati anche gli obiettivi opzionali: la dashboard real-time su Grafana e la tolleranza ai guasti tramite checkpointing.

II. ARCHITETTURA DEL SISTEMA

L’architettura realizzata per il *Progetto 2: Analisi in tempo reale di dati sui voli con Apache Flink* è una pipeline di *stream processing* organizzata secondo una topologia ben precisa, il cui framework di elaborazione è **Apache Flink**, attorno al quale ruotano i componenti di ingestione, storage e visualizzazione. La topologia complessiva è riportata in Figura 1.

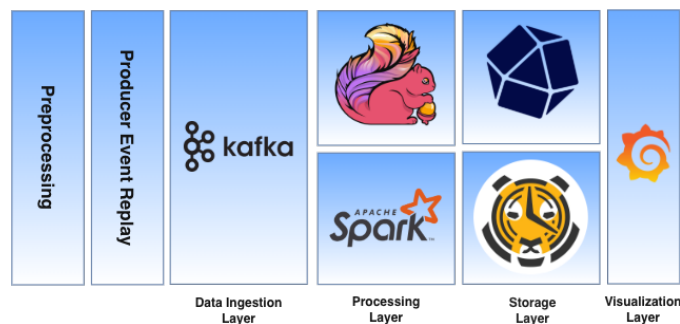


Figura 1. Architettura del sistema: preprocessing, replay accelerato, ingestione su Kafka con serializzazione Avro/Schema Registry, elaborazione in streaming su Flink (e Spark come confronto), CSV, InfluxDB/TimescaleDB, Grafana.

La pipeline è organizzata nei seguenti stadi logici:

- 1) **Preprocessing**. Il dataset CSV mensili viene letto, tipizzate e arricchite dell’*event time* di ciascun volo per poi essere ordinato temporalmente, producendo un unico file preparato pronto per il replay.
- 2) **Ingestion su Apache Kafka**. Un cluster Kafka a due broker in modalità *KRaft* espone il topic *flights* su cui un *producer* dedicato riproduce gli eventi, serializzandoli in formato **Apache Avro** con l’ausilio di un

Confluent Schema Registry. Il topic è partizionato (4 partizioni) per consentire il consumo parallelo.

- 3) **Stream processing su Apache Flink.** Un cluster Flink composto da un *JobManager* e un *TaskManager* (4 *task slot*, parallelismo 4) consuma il topic, assegna a ogni evento il timestamp logico, calcola le tre query su finestre basate sull'event time ed emette i *watermark*; lo stato è reso tollerante ai guasti dal checkpointing.
- 4) **Sink dei risultati.** L'output è prodotto in formato CSV e, in parallelo, due sink alimentano la dashboard di visualizzazione, tramite un topic specifico esposto dal cluster Kafka. La sorgente Kafka viene letta una sola volta e CSV, InfluxDB e TimescaleDB ricevono *le stesse identiche righe*.
- 5) **Visualizzazione.** Grafana mostra l'andamento temporale delle metriche prodotte dalle query.

Al fine di confronto, come già accennato, l'intera soluzione è stata inoltre reimplementata in **Apache Spark Structured Streaming**, che consuma lo stesso topic Kafka e produce output CSV analoghi separatamente.

L'intero sistema è containerizzato e orchestrato tramite **Docker Compose**, in continuità con l'approccio di deployment adottato nel anche *Progetto 1: Analisi di dati sui voli con Apache Spark*.

III. DATA INGESTION

La fase di data ingestion trasforma le sorgenti CSV grezze in uno stream di eventi ordinati e serializzati, consumabile dall'applicazione di stream processing. In particolare, questa fase si compone del preprocessing dei dati, del *producer* che effettua il replay e del trasporto su Apache Kafka con serializzazione Avro.

A. Preprocessing e dati preparati

Il dataset è fornito come collezione di file CSV, uno per ciascun mese di osservazione (da gennaio ad aprile 2025). Il preprocessing, che include la pulizia e la normalizzazione dei dati, viene eseguito in maniera disaccoppiata dal loro replay, rendendo di fatto quest'ultimo riproducibile e indipendente dal costo di parsing delle sorgenti.

1) *Selezione e tipizzazione dei campi:* Dalle sorgenti CSV vengono lette le sole 16 colonne utili alle query, scartando tutte le altre. Questi campi vengono quindi tipizzati e, a tal proposito, vogliamo porre l'attenzione sui valori numerici mancanti: la specifica consente di trattarli come null oppure come zero; così come nel *Progetto 1* è stata adottata la politica di lasciarli null, poiché un ritardo o una causa di ritardo assente è un dato *non osservato*, non un valore pari a zero. Questa politica si va a combinare con l'interpretazione di questi valori all'interno del calcolo delle query, in cui le medie ignorano automaticamente i null e i predicati booleani su valori null risultano falsi, evitando di alterare le statistiche sui ritardi.

2) *Calcolo dell'event time e ordinamento:* L'event time di ciascun volo è ricavato combinando i campi YEAR, MONTH, DAY_OF_MONTH e CRS_DEP_TIME (orario schedulato di partenza), come richiesto dalla specifica. Il timestamp è codificato in millisecondi dall'epoch (convenzione di Flink) e gestisce i casi limite dell'orario (ad es. il valore 2400 normalizzato a mezzanotte) e le date impossibili tramite un *fallback* al primo del mese. Il dataset viene infine *ordinato in modo stabile* per event time crescente: il replay parte quindi da una sequenza temporalmente ordinata, e l'eventuale **disordine** osservato a valle è esclusivamente *iniettato in modo controllato* dal producer, a fini sperimentali per le esecuzioni opportune, non un derivato dell'ordine originale dei file. Quindi il modulo di preprocessing genera un unico file CSV ordinato, memorizzato su un volume Docker condiviso anche con il modulo producer.

B. Apache Kafka

Il trasporto degli eventi è affidato ad **Apache Kafka**, un sistema di messaggistica distribuito basato su log append-only e partizionato. Il cluster è composto da **due broker** in modalità *KRaft*, in cui il consenso è gestito internamente da Kafka senza ricorrere ad Apache ZooKeeper: un nodo assume i ruoli di broker e controller, il secondo è broker-only e serve ad abilitare la replicazione (fattore di replicazione di default pari a 2). Gli eventi sono pubblicati sul topic *flights*, creato con **4 partizioni**: il partizionamento abilita il consumo parallelo da parte dei task di Flink e Spark. Un servizio ausiliario *Kafka UI* è incluso per il monitoraggio e il debugging del cluster.

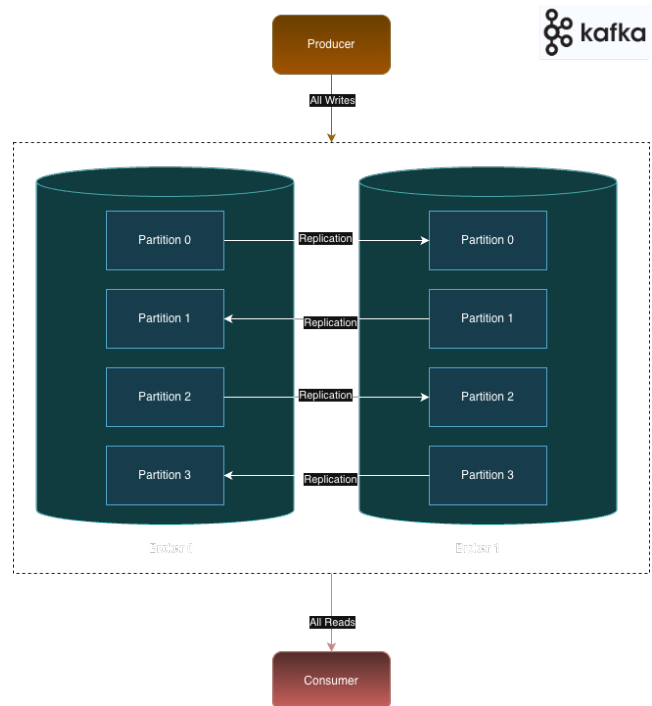


Figura 2. Kafka Cluster

C. Serializzazione Avro e Schema Registry

Gli eventi sono serializzati su Kafka in formato **Apache Avro**, un formato compatto e tipizzato, adatto allo scambio di record su uno stream. In maniera simile a quanto affrontato nel *Progetto 1: Analisi di dati sui voli con Apache Spark*, Avro è un tipo di formato basato su uno schema che definisce la tipizzazione. Nel particolare lo schema dei record `flight.avsc` va a definire la tipizzazione del messaggio `Flight`, gestendola in maniera coerente con la politica `null` del preprocessing.

Di particolare importanza la *gestione centralizzata* dello schema tramite un **Confluent Schema Registry**, un servizio che memorizza le versioni degli schemi e assegna a ciascuno un identificativo numerico. In fase di avvio, un servizio one-shot registra lo schema `flight.avsc`. I messaggi seguono il *Confluent wire format*: un *magic byte* (`0x00`), seguito dall'ID dello schema, seguito a sua volta dal *payload Avro binario*. Questo formato consente al consumer di risolvere lo schema tramite il registry e decodificare il record senza doverlo trasmettere a ogni messaggio. Il connettore `avro-confluent` di Flink e la funzione `from_avro` di Spark leggono direttamente da questo formato, risolvendo lo schema dal registry.

D. Producer

Il *producer* realizza la fase di **replay**: legge il file preparato e pubblica gli eventi sul topic `flights`, codificando ciascun record nel *Confluent wire format* descritto sopra.

1) *Accelerazione temporale*: Il producer applica un'accelerazione temporale che comprime in modo uniforme la scala dei tempi, preservando le relazioni temporali relative tra gli eventi: il ritardo di attesa prima dell'invio di ciascun evento è proporzionale alla distanza, in event time, dal primo evento, divisa per il fattore di accelerazione. Questo perché replicare gli eventi alla velocità originale richiederebbe l'intero arco temporale del dataset, circa 120 giorni. È stato adottato un fattore pari a **57 600×**: con esso un'ora di event time corrisponde a 0,0625 s di tempo reale, un giorno a 1,5 s e l'intero dataset viene riprodotto in circa **3 minuti**. Tale valore costituisce un compromesso fra la fedeltà alla dinamica temporale dei dati e la praticità sperimentale: il replay è abbastanza lungo da rendere osservabile il comportamento delle finestre e dei watermark in regime di streaming, ma sufficientemente rapido da permettere numerose esecuzioni ripetute durante la valutazione.

2) *Iniezione del disordine (out-of-orderness)*: Per valutare l'impatto della completezza dei risultati sulla latenza del livello di processamento, il producer può introdurre **eventi fuori ordine (out-of-order)**. Il disordine è simulato ritardando la consegna di una frazione di eventi, **senza mai modificarne l'event time**: un evento "trattenuto" conserva il proprio timestamp reale, ma viene inviato più tardi, comparando nel log Kafka dopo eventi a lui successivi.

Operativamente, con una certa probabilità p un evento viene posto in un *heap* ordinato per istante di consegna pianificato (in ritardo rispetto all'event time originale) e rilasciato solo quando il tempo di replay raggiunge tale istante. L'entità del ritardo è campionata da una distribuzione configurabile, *uniforme* oppure *esponenziale*, con un tetto massimo definito. Quest'ultimo è espresso in secondi di event time e rappresenta il *bound di out-of-orderness* D con cui confrontare il watermark: se il ritardo ammesso dal watermark è $\geq D$, gli eventi ritardati rientrano comunque nella finestra corretta; in caso contrario alcuni arrivano troppo tardi e vengono scartati come *late*. Per garantire la riproducibilità degli esperimenti il campionamento è governato da un seed fisso, e il parametro `max_in_flight_requests_per_connection` è posto a 1 per preservare l'ordine relativo degli eventi non trattenuti.

3) *Chiusura dello stream*: Poiché lo stream è finito, trattandosi di un replay di un dataset costruito precedentemente, l'ultima finestra event time non verrebbe chiusa spontaneamente in assenza di nuovi eventi che ne facciano avanzare il watermark. Per drenarla in modo automatico, al termine del replay il producer emette su *ogni* partizione del topic un *marker* di fine stream con event time collocato molto nel futuro rispetto all'anno di riferimento del dataset: abbiamo optato per l'anno 2200. Ciò spinge tutti i watermark per-partizione oltre l'ultima finestra reale, che viene così calcolata ed emessa senza ricorrere al comando manuale di drenaggio di Flink. Il marker è identificato dal codice compagnia speciale `__EOS__` ed è quindi escluso da tutte le query. In alternativa, l'arresto ordinato tramite il drenaggio del job produce il medesimo effetto.

IV. STREAM PROCESSING CON APACHE FLINK

A. Apache Flink

Apache Flink è un motore di elaborazione distribuita di stream, che tratta i dati come flussi illimitati elaborati a bassa latenza con stato gestito e tolleranza ai guasti. Il cluster utilizzato è composto da un *JobManager*, che coordina il submit e la schedulazione dei job, e da un *TaskManager* che ne esegue i task su 4 *task slot* (parallelismo di default pari a 4). Flink distingue il *tempo di elaborazione* dall'*event time*: le query di questo progetto operano interamente in event time, ricavato dal timestamp logico assegnato a ciascun evento, così da produrre risultati deterministici e indipendenti dalla velocità di arrivo. L'avanzamento del tempo è tracciato dai *watermark*: un watermark di valore t dichiara che non si attendono più eventi con timestamp inferiore a t , e autorizza la chiusura delle finestre il cui bordo è stato superato. La soglia di ritardo ammissibile del watermark rispetto al fronte dei dati costituisce il budget di attesa massima per gli eventi fuori ordine. Il checkpointing periodico garantisce la consistenza dello stato in caso di guasto.

I job sfruttano sia la *Table API/SQL*, di più alto livello e ottimizzata dal planner, sia la *DataStream API*, di più basso livello. Le tre query adottano un approccio ibrido, combinando le due API secondo l'operazione più naturale in ciascuno stadio.

1) *Eventi fuori ordine e watermarking*: È stato implementato un meccanismo di rilevamento di eventi fuori ordine e si confrontano più configurazioni di watermarking. Il disordine è generato dal producer ritardando la consegna di una quota di eventi entro un **limite massimo** D , come già introdotto nella sezione relativa al producer. La vera manopola che governa il compromesso tra *completezza* dei risultati e *latenza* del livello di processamento è il **ritardo massimo ammesso dal watermark** d . Un d ampio attende più a lungo gli eventi fuori ordine (completezza elevata, latenza alta), mentre un d ridotto chiude prima le finestre ma rischia di scartare eventi in ritardo (latenza bassa, possibile perdita). Sotto disordine uniforme in $[0, D]$ e trascurando i gap di quantizzazione del watermark, la frazione attesa di eventi persi in una finestra di ampiezza W vale $\text{loss} = p(D - d)^2 / (2DW)$: nulla quando $d \geq D$ (configurazione *safe*), massima quando $d = 0$ (configurazione *aggressive*).

Due elementi influenzano la *quantizzazione* con cui il watermark viene effettivamente applicato:

- l'intervallo di emissione periodica del watermark stesso (auto-watermark-interval)
- Per gli operatori Python, la durata del *bundle* (`python.fn-execution.bundle.time`), poiché i watermark sono applicati solo al confine del bundle.

Col valore di default di questi due parametri di configurazione, l'accelerazione dilaterrebbe a tal punto il loro equivalente in event-time da gonfiare il ritardo con cui avanza il watermark, permettendo anche a eventi arrivati in ritardo di entrare all'interno della finestra stessa. Infatti, i valori default (dell'ordine di 200 ms per l'emissione periodica del watermark e 1 s per il bundle time) e un fattore di accelerazione $57.600\times$ si traducono in salti enormi di event-time (200 ms $\rightarrow$ 3,2 h, 1 s di bundle $\rightarrow$ 16 h). Tra un salto e l'altro il watermark resta indietro per lunghi tratti di event-time, e finché non lo supera la finestra non scatta e non viene chiusa. Il risultato è che un evento fuori ordine trova ancora aperta la propria finestra e viene incluso, invece di essere scartato. Quindi negli esperimenti di completezza entrambi sono ridotti per rendere trascurabile il gap virtuale introdotto e isolare l'effetto di d . Il numero di eventi scartati perché arrivati oltre il watermark è misurato direttamente dalla metrica Flink `numLateRecordsDropped`, raccolta per job via REST API.

2) *Tolleranza ai guasti e Checkpointing*: Per la gestione della tolleranza ai guasti degli operatori con stato mediante il meccanismo di *checkpointing* di Flink, a intervalli regolari (10 s) Flink esegue uno snapshot coerente (*barrier-aligned*) dello stato di *tutti* gli operatori del job insieme agli offset

Kafka della sorgente; in caso di guasto, il job viene riavviato e ripristinato dall'ultimo checkpoint completato, mentre la sorgente Kafka riparte dagli offset salvati, senza perdita né duplicazione nei risultati.

Il meccanismo è applicato a tutti i job Flink:

- Modalità **EXACTLY_ONCE** (barrier allineate) con `max_concurrent=1`;
- Pausa minima tra checkpoint (5 s) per garantire progresso all'elaborazione, timeout di checkpoint (60 s) e numero di fallimenti consecutivi tollerati (3) prima di far fallire il job Flink;
- Checkpoint **esternalizzati**: l'ultimo checkpoint sopravvive a cancellazione/guasto ed è riutilizzabile per il recovery (`RETAIN_ON_CANCELLATION`);
- *Restart strategy* a ritardo fisso (fino a 10 tentativi, 10 s di attesa ciascuno): senza di essa il job fallirebbe invece di ripristinarsi.

3) *Query Q1*: La query Q1 realizza il monitoraggio in tempo reale dello stato operativo delle principali compagnie aeree. Facendo riferimento alle sole compagnie aeree **AA** (American Airlines), **DL** (Delta), **UA** (United) e **WN** (Southwest), gli eventi sono aggregati su *finestre tumbling di durata 1 ora basate sull'event time*. Per ciascuna finestra e per ciascuna compagnia la query calcola:

- il numero totale di voli osservati
- il numero di voli completati (non cancellati e non deviati), cancellati e deviati
- il valor medio di `DEP_DELAY` sui soli voli non cancellati
- il tasso di cancellazione (percentuale di voli cancellati sul totale della finestra)
- e il tasso di partenze in ritardo (percentuale di voli non cancellati con `DEP_DELAY` maggiore di 15 minuti).

La query è espressa con la *Table API/SQL*. Le compagnie di interesse sono filtrate *prima* del windowing, così che Flink non assegni alle finestre eventi di compagnie irrilevanti. L'aggregazione utilizza la *windowing TVF* `TUMBLE` su finestre di un'ora e raggruppa per finestra e compagnia. La gestione dei valori mancanti è coerente con la politica `null: COALESCE(cancelled, 0)` (e analogamente `diverted`) interpreta lo stato assente come "non cancellato/non deviato", mentre `AVG(dep_delay)` ignora automaticamente le righe con ritardo `null`. Il tasso di cancellazione è calcolato su *tutti* i voli della finestra, mentre il tasso di partenze in ritardo è normalizzato sui soli voli non cancellati, come da specifica.

L'aggregazione è materializzata nella vista `q1_agg`, unica fonte di verità: il sink CSV (sempre attivo) e i sink della dashboard leggono tutti la stessa vista, e gli `INSERT` sono raccolti in un unico `StatementSet`, ottenendo un solo job in cui la sorgente è letta una volta sola.

4) *Query Q2*: La query Q2 identifica in tempo reale gli aeroporti di partenza maggiormente interessati da ritardi significativi. Considerando tutti i voli non cancellati e non deviati,

un volo presenta un *ritardo significativo* se `DEP_DELAY` è maggiore di 30 minuti. La query è calcolata su tre finestre event-time: **1 ora, 6 ore e dall’inizio del dataset**. Per ciascuna finestra si produce la classifica aggiornata dei primi 10 aeroporti per numero decrescente di voli con ritardo significativo, considerando solo gli aeroporti con almeno 30 voli non cancellati e non devianti.

La query è espressa *interamente con la Table API/SQL* ed è organizzata, per ciascuna finestra, in due stadi logici:

- **Stadio 1 - aggregazione per aeroporto.** I voli non cancellati e non devianti sono pre-filtrati nella vista `completed_flights`; una *windowing TVF* `TUMBLE` raggruppa per finestra e aeroporto e calcola il numero di voli, quelli con ritardo significativo (`FILTER (WHERE dep_delay > 30)`), la media e il massimo del ritardo. La clausola `HAVING COUNT(*) >= 30` esclude gli aeroporti poco significativi. La lista dei voli in ritardo è raccolta con l’aggregato nativo `ARRAY_AGG(...)` `FILTER (WHERE dep_delay > 30)`; una funzione scalare Python (`format_top_delayed`, decoratore `@udf`) la ordina per ritardo decrescente e ne serializza i 20 voli peggiori.
- **Stadio 2 - classifica top-10.** Sulla vista aggregata, la funzione finestra `SQL ROW_NUMBER() OVER (PARTITION BY window_start, window_end ORDER BY severe_delays DESC, dep_delay_mean DESC, origin_airport_id ASC)` assegna il rank all’interno di ogni finestra completata, e la condizione `rn <= 10` trattiene i primi 10 aeroporti. L’output è *append-only* e quindi compatibile con il sink CSV.

La finestra “dall’inizio del dataset” è realizzata come una tumbling da 365 giorni: l’intero periodo gennaio-aprile 2025 ricade in un unico bucket, che si chiude quando il marker EOS spinge il watermark oltre il suo bordo. Come per Q1, tutti gli `INSERT` (sink CSV sempre attivo e, opzionalmente, i sink della dashboard) sono raccolti in un unico `StatementSet`, ottenendo un solo job in cui la sorgente è letta una volta sola.

5) *Query Q3:* La query Q3 calcola in tempo reale la distribuzione dei ritardi in partenza per compagnia e fascia oraria. Facendo riferimento alle compagnie aeree AA, DL, UA e WN e ai soli voli non cancellati e non devianti con `DEP_DELAY` presente, per ciascuna compagnia e per ciascuna delle 24 fasce orarie (ricavate da `CRS_DEP_TIME`) si calcolano il numero di voli, i percentili 25/50/75/90, il minimo e il massimo di `DEP_DELAY`. La query è calcolata su tre finestre event-time: **1 giorno, 7 giorni e dall’inizio del dataset**.

6) *Stima approssimata dei percentili con DDSketch in Q3:* La specifica richiede di calcolare i percentili dei ritardi alla partenza (`DEP_DELAY`) in tempo reale, senza ordinare né accumulare interamente i valori, mediante un algoritmo ap-

prossimato. È stato adottato **DDSketch** [1], uno sketch dei quantili con garanzia di *errore relativo massimo garantito* α , implementato in Python puro. L’asse dei valori è partizionato in bucket a spaziatura logaritmica: questo vuol dire che l’algoritmo mappa un qualunque valore $v > 0$ nel bucket di indice $\lceil \log_\gamma v \rceil$, dove $\gamma = (1 + \alpha)/(1 - \alpha)$ è il ratio del bucket, e ogni bucket mantiene solo un contatore intero. Per una query di quantile mappata al bucket di indice i , il valore restituito è il rappresentante

$$\tilde{v} = \frac{2\gamma^i}{\gamma + 1},$$

che garantisce un errore relativo $|\tilde{v} - v|/v \leq \alpha$ su tutto l’intervallo $(\gamma^{i-1}, \gamma^i]$ coperto dal bucket. Il quantile q si ottiene scorrendo i contatori in ordine crescente di valore fino alla posizione $q(n - 1)$.

Poiché i ritardi possono essere negativi (voli in anticipo), si usano due store speculari (uno per i valori positivi e uno per i valori negativi mappati su $|v|$) più un contatore per gli zeri. Minimo, massimo e conteggio sono mantenuti *esatti*, come richiesto. Due sketch si fondono sommando i contatori omologhi *fully mergeable*, proprietà che li rende compatibili con la `merge()` delle finestre di Flink.

Nella nostra implementazione lo store non è vincolato a un numero massimo di bucket, dato che il dominio dei ritardi è intrinsecamente ristretto, il numero di contatori resta ridotto senza bisogno di collasso e l’errore α è preservato sull’intero supporto. Con $\alpha = 0,01$ (da cui $\gamma \approx 1,0202$) e ritardi osservati nell’intervallo $[-50, +3403]$ minuti, ogni gruppo richiede al più $\lceil \log_\gamma 3403 \rceil \approx 407$ contatori sul lato positivo e ≈ 196 su quello negativo: *alcune centinaia* di interi, contro le decine di migliaia di valori grezzi che una finestra globale accumulerebbe per gruppo.

7) *Pipeline ibrida di Q3:* Il job combina le due API come Q1/Q2. La sorgente Kafka è definita come tabella con formato `avro-confluent` (deserializzazione via Schema Registry) e convertita in un `DataStream` con `to_data_stream`; su tale flusso avviene la *computazione* in `DataStream API`: assegnazione dei watermark *prima* del filtro sulle compagnie - così che il marker EOS avanzi il watermark e chiuda le finestre finali - `keyBy` per coppia compagnia/fascia oraria e finestra con `AggregateFunction` il cui accumulatore è un `DDSketch`. Il *sink* avviene in `Table API`, con lo stesso pattern `StatementSet` verso CSV e dashboard.

V. STREAM PROCESSING CON APACHE SPARK STRUCTURED STREAMING

A. Apache Spark Structured Streaming

A fini di confronto tra motori, le tre query sono state reimplementate in **Apache Spark Structured Streaming**, che consuma lo stesso topic Kafka e ne decodifica i messaggi, rimuovendo i byte relativi all’instanziazione del *Confluent wire format*. Ogni query adotta lo stesso modello di finestre event-time delle controparti Flink, dichiarate con `F.window`

e con `withWatermark` per la gestione dei disordini. In maniera simile a quanto visto nel *Progetto 1: Analisi di dati sui voli con Apache Spark*, nella soluzione Spark per il calcolo dei percentili di Q3 impieghiamo la funzione nativa `percentile_approx`, anch'essa approssimata, con un certo parametro di accuratezza. Essendo il replay finito, un controllo di inattività arresta automaticamente i job Spark quando non arrivano più dati da Kafka per un intervallo configurato.

VI. STORAGE DELLE METRICHE: INFLUXDB E TIMESCALEDB

Oltre all'output CSV certificato, l'architettura prevede la persistenza delle metriche delle query su un sistema di storage che funge anche da sorgente per la visualizzazione (Opzionale 1). Sono stati realizzati **due backend alternativi e indipendenti**, mantenuti entrambi a fini di confronto: il sink CSV resta in ogni caso sempre attivo, e i due backend sono semplici *consumatori paralleli* delle medesime viste dei job, dunque non sostituiscono né possono contraddire l'output consegnato. È previsto un topic/tabella per ciascuna finestra di ogni query.

A. InfluxDB

InfluxDB è un database *time-series nativo*. Non disponendo di un connettore Flink diretto, l'integrazione avviene tramite un ponte: il job pubblica le righe su un topic Kafka dedicato da cui **Telegraf** le consuma e le scrive su InfluxDB.

B. TimescaleDB

TimescaleDB è un database relazionale *SQL* (PostgreSQL con estensione *time-series*). A differenza di InfluxDB dispone di un *sink JDBC nativo* di Flink, che scrive direttamente senza componenti intermedi.

C. Confronto

Tabella I
CONFRONTO TRA I DUE BACKEND DI STORAGE DELLE METRICHE.

	InfluxDB	TimescaleDB
Paradigma	time-series nativo	SQL + estensione TS
Connettore Flink	assente (Telegraf)	JDBC nativo
Componenti	Kafka+Telegraf+DB	solo DB
Idempotenza	<code>last()</code>	upsert su PK
Linguaggio Query	Flux	SQL

VII. VISUALIZZAZIONE: GRAFANA

La visualizzazione real-time delle metriche prodotte dalle query è affidata a **Grafana**, in continuità con la scelta adottata nel *Progetto 1: Analisi di dati sui voli con Apache Spark*. Grafana è condiviso fra i due backend: tramite *provisioning* dichiarativo configura automaticamente entrambi i *datasource* (InfluxDB con linguaggio Flux e TimescaleDB con SQL) e le rispettive dashboard per le tre query, così da poter confrontare le due soluzioni fianco a fianco durante la presentazione. I pannelli mostrano l'andamento temporale, finestra dopo finestra, delle metriche delle query. Poiché il timestamp dei

punti è l'event time della finestra, l'intervallo temporale delle dashboard è impostato sul periodo del dataset (gennaio–aprile 2025): durante il replay accelerato le curve si costruiscono progressivamente, dando evidenza visiva della natura streaming dell'elaborazione.

Per quanto riguarda la finestra globale, definita dall'inizio del dataset, abbiamo interpretato la finestra come dall'inizio alla fine del dataset, chiudendola una volta arrivato l'ultimo evento. Ciò significa che nella finestra globale l'evoluzione real-time delle metriche non può essere visualizzato, dato che essa emette il proprio risultato solo quando il watermark ne supera il bordo destro: la chiusura è innescata dal *marker di fine stream* iniettato dal producer, che porta il watermark oltre il confine della finestra.

Per completezza abbiamo anche implementato una *vista cumulativa live* a bassa latenza con snapshot intermedi, in modo da vedere l'evoluzione real-time delle metriche. La cadenza di emissione è regolata in event time in Q2 (un'istantanea ogni 45 minuti di event-time) e in tempo di elaborazione in Q3 (ogni 5 secondi). La vista cumulativa calcola dunque la stessa quantità della finestra globale, ma la espone come serie temporale che ne mostra l'evoluzione *real-time*, mentre la finestra globale ne fornisce il valore finale, certificato a stream concluso.

VIII. DEPLOYMENT

Il deployment è stato effettuato tramite containerizzazione dei nodi del sistema con **Docker**, simulando un cluster di macchine distinte. Ogni nodo dell'architettura corrisponde a un container collegato a una stessa Docker Network, e l'orchestrazione dell'intero cluster è gestita tramite **Docker Compose**: un file descrive in modo dichiarativo tutti i servizi (i due broker Kafka, lo Schema Registry, JobManager e TaskManager Flink, i job delle query, i job Spark, i backend di dashboard e Grafana), definendone i parametri di configurazione e la comunicazione tra i nodi. La configurazione applicativa è centralizzata in un file YAML montato nei container, che consente di variare i parametri (senza ricostruire le immagini. Gli esperimenti invece sono descritti da file di override che ne estendono la configurazione base.

Come nel *Progetto 1: Analisi di dati sui voli con Apache Spark*, eseguire tutti i container sulla stessa macchina porta i nodi a contendersi un insieme ristretto di risorse computazionali e ad avere latenze di comunicazione prossime allo zero, condizione lontana da un caso reale distribuito ma adeguata agli scopi didattici e di valutazione comparativa del progetto.

IX. VALUTAZIONE DEI RISULTATI

Sono stati valutati sperimentalmente l'impatto del parallelismo dell'applicazione a fronte della latenza di processamento e del throughput. Inoltre è stato anche valutato l'impatto della completezza rispetto alla latenza di processamento nel contesto del rilevamento di eventi fuori ordine, gestito tramite watermarking. A tal fine sono stati predisposti un insieme di *esperimenti* riproducibili e degli *strumenti* per raccoglierne le

metriche: i valori numerici, le tabelle e i grafici sono prodotti eseguendo tali esperimenti e raccogliendone le metriche tramite i suddetti strumenti. Oltre alla discussione sperimentale, in questa sezione discutiamo anche dei risultati delle query.

A. Strumenti di misura

Un runner unico (`scripts/run_experiment.ps1`) esegue una delle query su un certo motore (Flink o Spark), applicando una configurazione di esperimento. Le metriche sono raccolte da script dedicati che scrivono i risultati in diversi file CSV:

- **Strumento di misura delle Performance:** campiona via REST le metriche del job Flink, come l'*active time*, e ne ricava il *throughput* del vertice sorgente (in record/s) e le percentuali di *busy* e di *backpressure* dell'operatore più carico (per individuare il collo di bottiglia).
- **Strumento di misura della Completezza:** somma la metrica `numLateRecordsDropped` degli operatori finestra, ossia gli eventi scartati perché giunti oltre il watermark. In Q3 i record in ritardo vengono deviati a un *side output* omonimo alla metrica nativa di Flink, poiché a causa dell'utilizzo di DDSketch, il contatore nativo `numLateRecordsDropped` dell'operatore finestra resta a 0.

B. Esperimenti predisposti

Gli esperimenti descrivono lo scenario per quanto riguarda disordine, watermark, e parallelismo.

- **Baseline:** fornisce l'output di riferimento per il controllo di completezza, senza iniettare disordine.
- **Sweep di watermarking:** disordine uniforme con **limite massimo di ritardo D** pari all'**ampiezza della finestra W** della query, variando *solo* il **ritardo ammesso d** tra *aggressive* ($d = 0$), *intermedio* ($d = W/2$) e *safe* ($d = W$). Le configurazioni sono organizzate per finestra così da sondare il compromesso completezza/latenza dove risiedono le finestre di ciascuna query.
- **Sweep di parallelismo** (`perf_par1`, `perf_par2`, `perf_par4`): scenario baseline tenuto fisso, variando solo il parallelismo di Flink tra 1, 2 e 4 (numero di partizioni del topic), per isolarne l'effetto su throughput e latenza.

Tabella II

FINESTRE E QUERY CORRISPONDENTI. PER CIASCUNA FINESTRA $D = W$ E IL RITARDO AMMESSO d È VARIATO TRA 0 (*aggressive*), $W/2$ E W (*safe*).

Finestra	W [s]	d testati [s]	Query (finestra)
1h	3600	0 / 1800 / 3600	Q1, Q2 (1h)
6h	21600	0 / 10800 / 21600	Q2 (6h)
1d	86400	0 / 43200 / 86400	Q3 (1d)
7d	604800	0 / 302400 / 604800	Q3 (7d)

1) *Confronto Apache Flink e Apache Spark Structured Streaming:* Il confronto **Flink vs Spark** si ottiene eseguendo lo stesso scenario con i due motori.

C. Checkpointing: verifica sperimentale del recupero

La prova di **checkpointing** serve a fornire una verifica del corretto recupero dello stato a seguito di un guasto. Il recupero è verificato da uno script dedicato che riproduce uno scenario di *guasto di nodo*:

- Con il job Q1 in esecuzione e il producer avviato (rallentato, così che il guasto cada durante l'elaborazione), lo script attende via REST il completamento di almeno un checkpoint
- Registra il valore di `numRestarts` come baseline
- Quindi **uccide il container del TaskManager**, simulando un guasto
- Riavvia poi il TaskManager e ne attende la registrazione
- L'esito è considerato positivo se:
 - il job riporta un ripristino dello stato da checkpoint: il campo `latest.restored` viene valorizzato, con l'ID del checkpoint da cui è stato recuperato lo stato
 - e il contatore `numRestarts` è aumentato, evidenza dell'avvenuto riavvio

Lo script prevede anche una modalità *baseline* senza guasto, il cui output può essere confrontato con quello del run con guasto per verificare che i risultati coincidano (nessuna perdita né duplicazione). Inoltre, ciò permette un confronto della latenza di processamento con e senza guasto.

D. Discussione dei risultati

1) *Completezza vs Latenza (watermarking):* In questo studio valutiamo la completezza e latenza al variare di d , la soglia di ritardo ammessa dal watermark. Si hanno i seguenti parametri: $p = 0.3$, $D = W$, $N = 2\,229\,457$ record. La perdita attesa è stata calcolata con la formula $p(D-d)^2/(2DW)$, con $D = W$, dipende solo da d/W e vale 15.00% per $d = 0$ 3.75% per $d = W/2$ e 0% per $d = W$. In realtà la perdita attesa è il **limite worst-case**. È possibile che la perdita misurata resti al di sotto tale limite a causa di uno slack dovuto alla quantizzazione del watermarking e alla densità non uniforme del traffico.

Query	W	d [s]	Regime	LateDropped [record]	Record persi sulla popolazione di riferimento [%]	Attesi [%]	Lat. [ms]
Q1	1h	0	aggressivo	143 893	10.9	15.00	249
		1800	intermedio	25 513	1.93	3.75	265
		3600	completo	0	0.00	0.00	434
Q2	1h	0	aggressivo	241 324	10.82	15.00	400
		1800	intermedio	44 575	2.00	3.75	489
		3600	completo	0	0.00	0.00	318
Q2	6h	0	aggressivo	288 977	12.96	15.00	628
		10800	intermedio	65 428	2.93	3.75	175
		21600	completo	0	0.00	0.00	294
Q3	1d	0	aggressivo	139 790	10.59	15.00	n/d
		43200	intermedio	35 032	2.65	3.75	n/d
		86400	completo	0	0.00	0.00	n/d
Q3	7d	0	aggressivo	185 587	14.05	15.00	n/d
		302400	intermedio	29 559	2.24	3.75	n/d
		604800	completo	0	0.00	0.00	n/d

Tabella III
CONFRONTO COMPLETEZZA VS LATENZA

2) *Confronto Flink vs Spark*: Il confronto tra Apache Flink e Apache Spark Structured Streaming è stato condotto sulla stessa piattaforma, utilizzando una configurazione standard che prevede 4 partizioni per Apache Kafka e un livello di parallelismo impostato anch'esso a 4 per Flink.

Query	Flink (p4/n4)		Spark (p4/n4)		Spark ÷ Flink
	Throughput [rec/s]	Active time [s]	Throughput [rec/s]	Active time [s]	
Q1	51.9k	43.0	208.3k	10.7	4.0×
Q2	51.3k	43.5	91.7k	24.3	1.8×
Q3	24.3k	91.9	163.3k	13.6	6.7×

Tabella IV
CONFRONTO PRESTAZIONI FLINK VS SPARK

A una prima lettura dei dati, sui carichi di tipo compute-bound backlog, Spark domina nettamente, registrando prestazioni superiori che vanno da 2.8 a 6.7 volte rispetto al rivale. Tuttavia, è necessaria un'importante precisazione per contestualizzare questo risultato: questo è esattamente il terreno di gioco ideale per Spark. Drenare in blocco una coda (compute-bound backlog) equivale, di fatto, a eseguire un'operazione batch mascherata da streaming. In questo specifico scenario, l'architettura a micro-batch colonnare e la whole-stage code generation di Spark eccellono. Flink, al contrario, nasce con una filosofia diversa: è progettato per lo streaming continuo a bassissima latenza, non per smaltire massivamente un backlog. In un report tecnico, questo caveat è obbligatorio; in caso contrario, limitarsi a concludere che "Spark batte Flink" restituirebbe un'immagine parziale e fuorviante.

Emerge chiaramente che i numeri non riflettono solo una differenza tra i due engine, ma raccontano l'impatto diretto

delle scelte implementative:

- Le Query 1 e 2 (il costo del modello a record): In questi test, eseguiti utilizzando Flink SQL puro su JVM, il vantaggio di Spark varia da 1.8x (nella Query 2) a 4.0x (nella Query 1). Questo divario rappresenta il costo strutturale inevitabile dell'elaborazione record-by-record tipica dell'architettura di Flink, messa a confronto con i micro-batch di Spark.
- La Query 3 (il collo di bottiglia di Python): La situazione cambia drasticamente nella terza query, dove il divario esplode fino a un 6.7x in favore di Spark. Il motivo strutturale è evidente: è l'unico test eseguito in DataStream utilizzando Python. In questo caso subentra la pesante penalità introdotta da PyFlink. La gestione tramite process function per singolo record, unita ai continui costi di serializzazione tra l'ambiente Java e quello Python, finisce per dimezzare letteralmente il throughput di Flink rispetto a quello ottenuto nelle prime due query, crollando a 24.3k record/s contro i 51.9k e 51.3k rispettivamente di Q1 e Q2.

3) *Studio del parallelismo*: Per valutare la scalabilità delle tre query su Flink sono stati misurati il throughput medio (numero di record elaborati al secondo) e il tempo necessario a smaltire un medesimo carico di circa 2,23 milioni di record, facendo variare due parametri: il grado di parallelismo degli operatori (p) e il numero di partizioni del topic Kafka di ingresso (n). I risultati, riportati in Tabella V e illustrati in Figura ??, mettono in luce due comportamenti nettamente distinti.

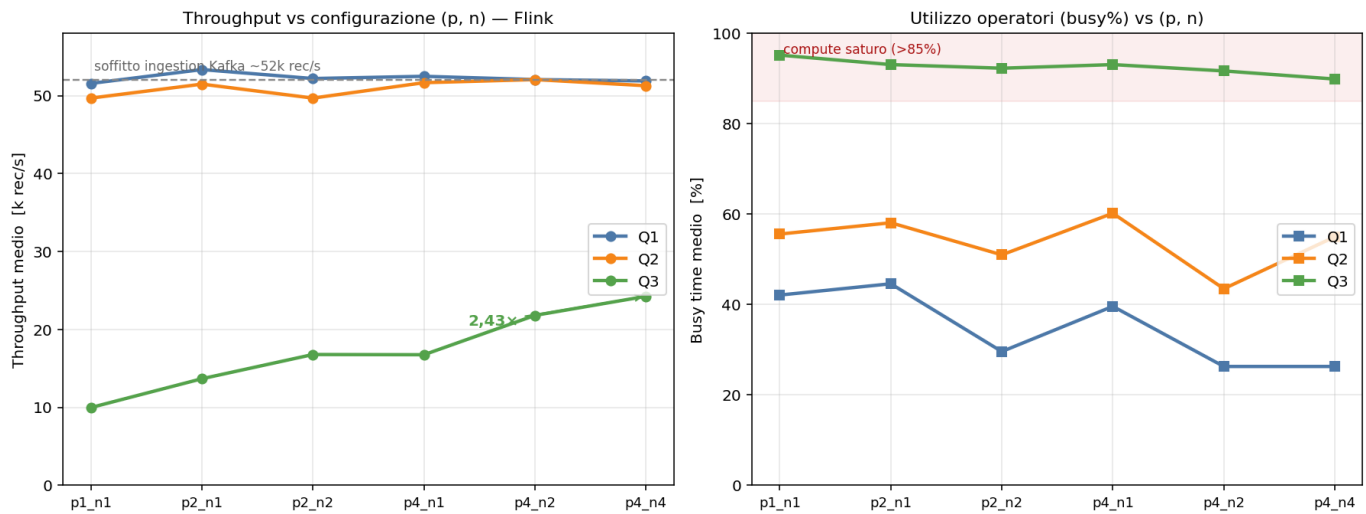


Figura 3. Throughput e Business rispetto alle configurazioni di parallelismo

Tabella V
STUDIO DEL PARALLELISMO SU FLINK

Config (p, n)	Throughput [k rec/s]	Tempo attivo [s]	Busy [%]	Backpr. [%]	Speedup (vs p1_n1)
Q1					
p1_n1	51,6	43,2	42,1	0,0	1,00×
p2_n1	53,4	41,8	44,6	0,0	1,03×
p2_n2	52,2	42,7	29,6	0,0	1,01×
p4_n1	52,5	42,5	39,6	0,0	1,02×
p4_n2	52,1	42,8	26,3	0,0	1,01×
p4_n4	51,9	43,0	26,3	0,0	1,01×
Q2					
p1_n1	49,7	44,9	55,6	7,2	1,00×
p2_n1	51,5	43,3	58,1	3,1	1,04×
p2_n2	49,7	44,9	51,0	6,2	1,00×
p4_n1	51,7	43,1	60,2	1,7	1,04×
p4_n2	52,1	42,8	43,5	3,6	1,05×
p4_n4	51,3	43,5	55,0	24,5	1,03×
Q3					
p1_n1	10,0	223,3	95,2	29,0	1,00×
p2_n1	13,7	163,0	93,1	8,0	1,37×
p2_n2	16,8	132,8	92,3	36,6	1,68×
p4_n1	16,8	133,0	93,1	0,0	1,68×
p4_n2	21,8	102,2	91,7	21,4	2,18×
p4_n4	24,3	91,9	89,9	43,9	2,43×

L'analisi mostra che l'aumento del parallelismo è una leva efficace unicamente per le query *compute-bound*, cioè nel nostro caso la sola Q3 e a condizione che il numero di partizioni sia adeguato. Per Q1 e Q2, il cui limite è imposto dalla velocità di ingestione, il miglioramento delle prestazioni non passa dall'incremento del parallelismo, quanto piuttosto da interventi sulla capacità di ingresso (un maggior numero di partizioni o di broker) o dall'adozione di un formato di deserializzazione più efficiente.

X. USO DI STRUMENTI DI AI GENERATIVA PER IL PROGETTO 2

Durante lo svolgimento del *Progetto 2: Analisi in tempo reale di dati sui voli con Apache Flink* i principali strumenti di AI generativa utilizzati sono stati: *ChatGPT* e *GitHub Copilot*. L'impiego di questi strumenti ha avuto l'obiettivo di velocizzare lo sviluppo progetto, soprattutto nelle fasi che non richiedessero uno sviluppo di una qualsiasi logica o dell'applicazione di metodologie apprese durante il Corso di Sistemi e Architetture per Big Data, come per esempio la configurazione dei servizi o framework utilizzati. Precisiamo che l'architettura del progetto è stata totalmente sviluppata e ragionata dai membri del gruppo e che gli strumenti di AI generativa possono essere stati utilizzati nella fase progettuale solo con un'accezione esplorativa verso alternative possibili rispetto a quanto deciso in prima istanza, o per la ricerca di un'ottimizzazione delle configurazioni dell'architettura sviluppata da noi. Ogni scelta critica è stata presa dai membri del gruppo dopo un'attenta analisi di quanto generato tramite questi strumenti e dopo un controllo ulteriore presso fonti ufficiali, quali le documentazioni dei servizi o framework utilizzati.

Gli strumenti sono stati utilizzati anche per una fase di pulizia del codice, in favore di una maggiore leggibilità dello stesso, andando a creare una ristrutturazione del codice che permet-

tesse anche un riutilizzo di funzioni tra moduli diversi, sempre grazie a prompt specifici pensati dai membri del gruppo.

Nello sviluppo del codice ci siamo avvalsi dell'aiuto di strumenti di AI generativa per l'implementazione di algoritmi specifici, in particolare: *DDSketch* nel calcolo dei percentili nella Query 3. Ci teniamo a precisare che l'utilizzo di questi strumenti in questa fase è stato da supporto per un controllo ulteriore sulla correttezza delle implementazioni sviluppate, più che nello sviluppo delle stesse. Anche in questo caso, ogni scelta critica è stata presa dai membri del gruppo dopo un'attenta analisi di quanto generato tramite questi strumenti e dopo un controllo ulteriore presso fonti ufficiali, quali i riferimenti forniteci assieme alla specifica per alcuni di questi algoritmi.

Per la fase di debugging durante lo sviluppo del progetto i membri del gruppo hanno ricorso agli strumenti di AI generativa, soprattutto nei casi in cui occorreva interpretare gli output di errori o warning restituiti dai servizi o framework utilizzati, in maniera tale da velocizzare il processo di individuazione dei problemi e permetterci quindi di agire in prima persona per la risoluzione degli stessi.

Il report presentato è stato scritto interamente dai membri del gruppo, senza avvalersi dell'utilizzo di strumenti di AI generativa per la stesura iniziale del testo. Non è avvenuto nemmeno un controllo sintattico e stilistico dei contenuti prodotti in maniera originale dai membri del gruppo, ma solo un controllo di coerenza del contenuto della relazione stessa con il codice sviluppato.

I membri del gruppo tengono a precisare di nuovo che ogni utilizzo degli strumenti generativi è stata utilizzata *da supporto* per lo sviluppo e non come *sostituto* dello stesso. Ogni riga di codice generata è stata analizzata e verificata prima di essere stata inserita manualmente all'interno del progetto.

RIFERIMENTI BIBLIOGRAFICI

- [1] C. Masson, J. E. Rim, and H. K. Lee, "DDSketch: A fast and fully-mergeable quantile sketch with relative-error guarantees," *Proc. VLDB Endowment*, vol. 12, no. 12, pp. 2195–2205, 2019.